



University of Milan
Data Science for Economics

Finding Similar Items

Algorithms for Massive Data Project

Mohammadhadi Shahhosseini
31812A

October 2025

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

Contents

1	Introduction	3
2	Data Description	3
3	Data Preprocessing	4
3.1	Data Organization	4
3.2	NLP Preprocessing	4
4	Methods and Implementation	5
4.1	Shingling (n-grams over tokens)	6
4.2	Binary Feature Hashing (HashingTF)	6
4.3	Similarity of Reviews	6
4.3.1	Jaccard Similarity	6
4.3.2	MinHash and its link to Jaccard	7
5	Scalability	7
6	Results and Discussion	8

1 Introduction

Online review platforms accumulate millions of short, noisy texts that often contain duplicated or near-duplicated content. Detecting such redundancy is useful for downstream analytics (e.g., opinion mining, recommendation, and quality control), and it is also a classical testbed for large-scale similarity search. In this project, I study near-duplicate detection on the *Amazon Books Reviews* dataset using a distributed pipeline built with PySpark, Spark NLP, and MinHashLSH.

The technical goal is to retrieve pairs of reviews that are highly similar in content without performing an infeasible all-pairs comparison. To that end, I first apply light normalization and tokenization, then construct tri-gram shingles to capture local phrase structure. Each review is embedded as a sparse, binary vector via the hashing trick (**HashingTF**), and candidate pairs are retrieved with MinHash locality-sensitive hashing. Jaccard similarity is used as the target measure; Spark returns Jaccard distance from the LSH join, which I convert to similarity as $1 - \text{distance}$.

2 Data Description

This project is based on the *Amazon Books Reviews*[1] dataset published on Kaggle under a CC0 (public domain) license. The file accessible with Kaggle API includes two datasets that I focused on *Books rating*. This dataset contains 3000000 user-generated reviews for books which provide realistic conditions for a locality-sensitive hashing (LSH) pipeline. I worked with the CSV dataset in a PySpark + Spark NLP environment on Google Colab.

Table 1 summarizes the key metadata. The dataset has roughly three million rows and ten columns. As is common with large user-generated collections, some fields contain missing values and various non-linguistic symbols. These properties motivate basic text cleaning and length filtering before similarity estimation.

Table 1: Dataset metadata for the Amazon Books Reviews corpus.

Attribute	Value
Dataset	Amazon Books Reviews (Kaggle)
License	CC0 (Public Domain)
Primary file	Books_rating.csv
Approx. rows	~3,000,000
Columns	10
Primary modality	Text (user reviews)
Task in this work	Near-duplicate detection on review text
Fields used	Id, Title, review/text $\rightarrow$ text
Execution setup	Google Colab, PySpark, Spark NLP

The original file includes ten columns per record (all read as **string** by Spark at load time). For the core text-based method, I focus on three fields: the unique identifier **Id**, the **Title** (for light context when inspecting examples), and the review body **review/text**. The review field is later renamed to **text** for clarity in the code. Other attributes (such as **Price**, **review/score**, and **review/time**) are not required by the MinHash-based similarity pipeline and are therefore not used in the main experiments. This choice helps isolate the effect of textual redundancy and keeps runtime predictable.

A snapshot of the raw schema (after loading) is shown in Table 2. Spark reports near-complete coverage for the text fields, while some non-text attributes (e.g., **Price**) are sparser. Since the methodology is purely text-based, completeness of **review/text** is the main requirement for correctness and replicability.

Table 2: Schema and non-null counts at load time. All columns are read as `string`.

Column	Type	Non-null count (approx.)
<code>Id</code>	string	3,000,000
<code>Title</code>	string	2,999,792
<code>Price</code>	string	482,421
<code>User_id</code>	string	2,437,750
<code>profileName</code>	string	2,437,800
<code>review/helpfulness</code>	string	2,999,633
<code>review/score</code>	string	2,999,870
<code>review/time</code>	string	2,999,973
<code>review/summary</code>	string	2,999,935
<code>review/text</code>	string	2,999,957

3 Data Preprocessing

3.1 Data Organization

I organize the corpus with a minimal set of columns required for text-based similarity. After loading `Books_rating.csv`, I select `Id`, `Title`, and `review/text`, and I rename `review/text` to `text` for clarity. Rows with nulls in these fields are removed. The remaining attributes are not used by the core pipeline and are dropped to keep memory and runtime predictable on Spark.

A key design choice concerns de-duplication. Since the task aims at detecting *near* duplicates rather than reproducing a long list of exact copies, I try to prevent trivial Jaccard scores of 1.0. I explored three strategies in sequence:

- **(A) De-duplicate by {Id, text}.** This assumes exact duplicates occur only within the same book record. In practice, it removes exact re-posts under the same `Id`, but it does not account for different editions or catalog entries of the same book.
- **(B) De-duplicate by {Title, text}.** This extends (A) to clean exact copies within a title. However, the dataset contains multiple editions and variants of the same work, often with slightly different titles. Moreover, the same review text appears across different titles, so exact copies still leak through.
- **(C) De-duplicate by {text} only (final choice).** This eliminates exact textual duplicates globally, independent of `Id` or `Title`. It best matches the objective: keep the dataset free of verbatim copies so that the LSH stage focuses on *similar* (not identical) reviews.

The evidence for (A) and (B) being insufficient comes from simple frequency checks on the raw and normalized text. For example:

- On raw text: `reviews_df_clean.groupBy("review/text").count().orderBy(desc("count"))` shows many repeated strings, e.g., “*digital books are perfect and easy to use! ...*” (321 occurrences) and even single-token entries like “*5.0*” (425 occurrences).
- On normalized text: `df.groupBy("text").count().orderBy(desc("count"))` still reveals frequent duplicates, e.g., “*building up my library so no time to read then review ...*” (221 occurrences).

3.2 NLP Preprocessing

I prepare the review texts with a light but effective pipeline that reduces noise, enforces minimum content, and standardizes tokens for shingling and LSH. The goal is to keep the text close to its original form while removing factors that inflate trivial matches.

Text normalization (Spark SQL). Starting from the selected column `text`, I apply: (i) lowercasing; (ii) Unicode-aware removal of non-letters/digits using `\p{L}` and `\p{Nd}`, replacing them with a space; (iii) whitespace compaction to a single space; (iv) a short-text filter with `text_len ≥ 5` to discard very small entries (e.g., “5.0”); and (v) a global exact de-duplication on `text`. This sequence stabilizes token statistics and prevents a flood of pairs with Jaccard= 1.

Flexible sampling for development. To iterate quickly while preserving the full pipeline logic, I expose a sampling switch in `nlp_clean`: `use_sample` toggles between the entire dataset and a development subset; `sample_mode` allows either a fixed number of rows (random order with a seed) or a fixed fraction of the data. The seed (`sample_seed`) makes runs repeatable. In final experiments, the same function runs on the full dataset by setting `use_sample=False`.

Spark NLP pipeline. I implement a compact, reproducible pipeline:

1. **DocumentAssembler** (`text` → `document`): wraps raw text as a Spark NLP document.
2. **Tokenizer** (`document` → `token`): splits text into tokens.
3. **Normalizer** (`token` → `normalized`): lowercases and removes residual non-alphanumerics via `setCleanupPatterns(["^A-Za-z0-9"])`.
4. **StopWordsCleaner** (`normalized` → `cleanTokens`): removes common stopwords (case-insensitive).
5. **LemmatizerModel** (`cleanTokens` → `lemma`): uses the pretrained `lemma_antbnc` model (`en`) to reduce inflected forms.
6. **Finisher** (`lemma` → `final_tokens`): outputs clean Python arrays for downstream Spark ML.

After transformation, I filter out short token sequences by requiring `size(final_tokens) ≥ min_tokens` (default = 3), then keep only `Id`, `text`, and `final_tokens`. To support scalability and repeated passes in the experiments, the resulting dataframe is cached in memory and materialized once with a forced `count()`.

Table 3: Example of tokenized book review data

Id	text	final_tokens
0312033613	1939 is hailed as...	[1939, hail, one, ...]
B0006E2W9M	set during the g...	[set, great, depr...]
0570053641	nikki rach writes...	[nikki, rach, wri...]
1562533703	the book is very...	[book, helpful, q...]
0613066421	this was the thir...	[third, binchy, n...]

Overall, this preprocessing keeps the corpus *lightly* cleaned (no heavy normalization or semantic rewriting), ensures reproducibility via seeded sampling, and produces a standardized token field (`final_tokens`) that is suitable for tri-gram shingling and MinHash LSH in the next section.

4 Methods and Implementation

The pipeline follows a classical set-similarity approach tailored to large-scale text: I convert each review into a set of characterizing tokens (shingles), embed these sets into sparse binary vectors via feature hashing, and approximate Jaccard similarity through MinHash LSH for efficient candidate generation and joining.

4.1 Shingling (n-grams over tokens)

Let $\mathcal{T}(d) = (t_1, t_2, \dots, t_m)$ be the token sequence of a review d after preprocessing. I form tri-gram shingles[2]

$$S(d) = \{(t_i, t_{i+1}, t_{i+2}) : 1 \leq i \leq m - 2\}.$$

Shingles incorporate limited local order and context, making exact copies and lightly paraphrased texts more likely to share elements than bag-of-words alone. In practice, I use Spark ML’s NGram(n=3) over the `final_tokens` field, yielding a variable-size set $S(d)$ per document. Tri-grams are a compromise: $n=2$ tends to over-match (high recall, lower precision), while $n>3$ increases sparsity and memory.

Table 4: Example of tokenized data with shingles

Id	text	final_tokens	Shingles
0312033613	1939 is hailed as...	[1939, hail, one, ...]	[1939 hail one, h...]
B0006E2W9M	set during the g...	[set, great, depr...]	[set great depres...]
0570053641	nikki rach writes...	[nikki, rach, wri...]	[nikki rach write...]
1562533703	the book is very...	[book, helpful, q...]	[book helpful que...]
0613066421	this was the thir...	[third, binchy, n...]	[third binchy nov...]

4.2 Binary Feature Hashing (HashingTF)

I map $S(d)$ into a high-dimensional, extremely sparse binary vector $x(d) \in \{0, 1\}^D$ using the hashing trick:

$$\phi : \text{shingle} \mapsto \{0, \dots, D - 1\}, \quad x_j(d) = \begin{cases} 1 & \exists s \in S(d) : \phi(s) = j, \\ 0 & \text{otherwise.} \end{cases}$$

I employ HashingTF with `binary=True` so that the representation encodes set membership (presence/absence) rather than counts, which is aligned with Jaccard’s set semantics. I set $D = 2^{18} = 262,144$ features. This dimensionality balances: (i) memory and shuffle costs (too large D increases index width and sparsity overhead), (ii) collision risk (too small D merges unrelated shingles). In practice, D in the range 2^{18} – 2^{20} works well for millions of short documents; I use 2^{18} to keep join sizes and broadcast plans stable under limited Colab resources.

Table 5: Example of feature extraction pipeline output

Id	text	final_tokens	Shingles	features
0312033613	1939 is hailed as...	[1939, hail, one, ...]	[1939 hail one, h...]	(262144,[331,1531...])
B0006E2W9	set during the g...	[set, great, depr...]	[set great depres...]	(262144,[4298,908...])
0570053641	nikki rach writes...	[nikki, rach, wri...]	[nikki rach write...]	(262144,[40526,44...])
1562533703	the book is very...	[book, helpful, q...]	[book helpful que...]	(262144,[2277,559...])
0613066421	this was the thir...	[third, binchy, n...]	[third binchy nov...]	(262144,[3902,542...])

4.3 Similarity of Reviews

4.3.1 Jaccard Similarity

Given two documents d_a, d_b with shingle sets S_a, S_b , the Jaccard similarity [2] is

$$J(S_a, S_b) = \frac{|S_a \cap S_b|}{|S_a \cup S_b|},$$

a standard measure of set overlap. Jaccard equals 1 only for identical sets and 0 when the sets are disjoint. Because shingles are set elements, binary HashingTF preserves the indicator structure needed by Jaccard (frequency is not required).

4.3.2 MinHash and its link to Jaccard

MinHash provides an unbiased estimator of Jaccard: for a single MinHash function h , the probability that the minima coincide equals the Jaccard similarity,

$$\Pr[\min_{s \in S_a} h(s) = \min_{s \in S_b} h(s)] = J(S_a, S_b).$$

With k independent hash functions, the estimator

$$\hat{J}(S_a, S_b) = \frac{1}{k} \sum_{r=1}^k \mathbf{1}[\min_{s \in S_a} h_r(s) = \min_{s \in S_b} h_r(s)]$$

satisfies $\mathbb{E}[\hat{J}] = J$ and $\text{Var}(\hat{J}) = J(1-J)/k$. This property underpins efficient locality-sensitive hashing (LSH): similar sets collide in the same hash buckets with higher probability, enabling sublinear candidate retrieval.

MinHash compresses each (potentially large) shingle set into a short signature of k integers such that the *collision rate* across positions is an unbiased estimate of Jaccard. LSH organizes these signatures so that only pairs likely to have high J are considered during the join, which avoids the $O(N^2)$ all-pairs comparison otherwise required. In our setting, tri-gram shingles capture local phrase structure; binary hashing preserves set membership at low memory; and MinHashLSH filters and scores candidates at scale with controllable accuracy via k and the similarity threshold τ .

I use MinHashLSH over the hashed binary vectors (**features**) and fit an index with **numHashTables** = k (I set $k=6$). Increasing k generally improves recall of true neighbors while also increasing index size and join time; decreasing k does the opposite. After fitting, I run

```
approxSimilarityJoin(df_hashed, df_hashed, 1- $\tau$ )
```

which returns candidate pairs whose estimated Jaccard distance is at most $1-\tau$ (i.e., similarity at least τ). I then remove self-pairs and symmetric duplicates using the document identifiers, and I compute the reported score as $1 - \text{jaccard_dist}$.

5 Scalability

The pipeline is designed to scale on distributed dataflows in Spark. Each stage either operates in a map-like fashion (embarrassingly parallel) or uses bounded shuffles that remain tractable for millions of short documents. Below, I outline the main cost drivers and the choices that keep the pipeline responsive under limited resources (e.g., Colab) while remaining correct on the full dataset.

Key parameters and trade-offs.

- **Shingle length** n . Smaller n increases overlap and the number of candidates (higher recall, more work). I set $n=3$ as a balance between precision and cost.
- **Hashing dimension** D . Larger D reduces collisions but increases index width and memory. I use $D = 2^{18}$ to keep shuffles light while limiting accidental merges of unrelated shingles.
- **MinHash tables** k . Increasing k improves neighbor recall and score stability but adds per-row signature work and larger LSH structures. I set $k=6$ to keep joins fast with acceptable accuracy.
- **Similarity threshold** τ . Higher τ yields fewer pairs and cheaper joins; lower τ expands candidate sets superlinearly. Tuning τ is the most direct lever on join size.

Table 6: Operational profile of major stages and their scalability characteristics.

Stage	Pattern	Dominant cost	Notes
Text normalization	Map	$O(N \cdot L)$	Per-row regex/lowercasing; no shuffle.
Spark NLP	Map	$O(N \cdot L)$	CPU-bound; cache output for reuse.
N-gram shingling	Map	$O(N \cdot L)$	Linear in token length L .
HashingTF (binary)	Map	$O(N \cdot S)$	Constant-time hash per shingle; sparse vectors.
MinHashLSH fit	Shuffle (bounded)	$O(N \cdot k)$	$k = \text{numHashTables}$; small per-doc signatures.
Approx. similarity join	Shuffle (filtered)	$O(N \log N)$ to $O(N^{1+\epsilon})$	Sub-quadratic; candidates pruned by LSH.

Memory and shuffle management. Two caches are decisive: (i) post-NLP tokens (`final_tokens`) and (ii) hashed features (`features`). Caching avoids recomputation across multiple passes (different τ or k). Because each review yields a very sparse vector, the memory footprint scales roughly with the number of *distinct* shingles rather than the Cartesian product of documents. The LSH fit/join stages introduce shuffles; their cost is bounded by signature size (k) and by the candidate filtering implied by τ .

Sampling vs. full runs. For development, the pipeline exposes a deterministic sampling switch (fixed-size or fractional). This keeps iteration times short without changing logic. Final measurements run with `use_sample=False`; no code changes are required, so results are replicable on the entire dataset.

Asymptotic behavior. In the absence of LSH, all-pairs Jaccard would be $O(N^2)$. With MinHashLSH, candidate generation becomes sub-quadratic, and the effective complexity is driven by the number of buckets hit per document (a function of k , D , and shingle statistics).

6 Results and Discussion

In this section, I present the review–review similarity results obtained after the full pipeline described earlier. By using *MinHash LSH* as an unbiased estimator of the Jaccard measure (Spark returns the Jaccard *distance*, which I convert to Jaccard *similarity* as $1 - \text{distance}$), I retrieve the pairs shown below. Table 7 reports the top eight matches.

Table 7: Top near-duplicate pairs (tri-grams, binary HashingTF with $D=2^{18}$, MinHashLSH with $k=6$)

id.a	id.b	$\hat{J}$	text.a	text.b
061858532X	B00005UVH9	0.286	this is the best book ...	one of the best books ...
0553471058	0889626286	0.286	this is a well written ...	this is one of the ...
0727858491	B00005UVH9	0.286	this is one of the ...	one of the best books ...
1574883488	B000KF8HTG	0.250	this book could also be ...	this review pertains to dandelion ...
B0007IWOGI	B000NWU3I4	0.231	we bought this as a ...	i read this book years ...
0889626286	B000HJNEYS	0.222	this is one of the ...	i have to say that ...
B00005UVH9	B000N6R4PU	0.190	one of the best books ...	a tremendous conclusion to the ...
0889626286	B000K0BJKU	0.188	this is one of the ...	this is by far the ...

Overall, the retrieved pairs show moderate estimated Jaccard similarity ($\hat{J} \approx 0.19\text{--}0.29$), consistent with short, templated praise and light paraphrases across editions and titles. Because exact textual duplicates were removed beforehand, these results focus on *near* duplicates rather than trivial matches, providing a practical basis for threshold and shingle-size tuning in downstream applications.

References

- [1] Mohamed Bakhet. Amazon books reviews. <https://www.kaggle.com/datasets/mohamedbakhet/amazon-books-reviews/data>, 2020.
- [2] Jure Leskovec, Anand Rajaraman, and Jeffrey David Ullman. *Mining of Massive Datasets*. Cambridge University Press, USA, 2nd edition, 2014.